

```

import asyncio
import asyncpg
import logging
from datetime import datetime

log = logging.getLogger(__name__)

def now():
    return datetime.utcnow()

async def backfill_outcome_types(conn):
    """
    One-time migration for resolved alerts that predate the outcome_type
    and return tracking columns. Safe to run on every startup – it only
    touches rows where outcome_type IS NULL and outcome IS NOT NULL.

    Derives:
        entry_price      – from yes_price (the price at alert time)
        exit_price       – from outcomePrices via Polymarket API (YES=0.99, NO=0.01
        exit_return_pct  – (exit - entry) / entry * 100
        peak_price       – best estimate: exit_price if win, entry_price if loss
                        (true peak requires price history; this is a safe approx
        peak_return_pct  – same basis as above
        outcome_type     – FULL_WIN / LOSS derived from outcome + entry_price
        exit_reason      – RESOLVED (all backfilled trades were held to resolution)
    """
    rows = await conn.fetch("""
        SELECT id, yes_price, outcome, profitable
        FROM alerts
        WHERE outcome IS NOT NULL
        AND outcome_type IS NULL
    """)

    if not rows:
        log.info("Backfill: nothing to migrate")
        return

    log.info("Backfilling %d resolved alerts with outcome_type + return data...",
        updated = 0

    for row in rows:
        try:
            entry = float(row["yes_price"])

```

```

outcome = row["outcome"]          # "YES", "NO", or "PARTIAL"
profitable = row["profitable"]

# Derive exit price from outcome
if outcome == "YES":
    exit_price = 0.99
elif outcome == "NO":
    exit_price = 0.01
else:
    # PARTIAL - we don't know exact exit, use entry as neutral fallback
    exit_price = entry

# Derive outcome_type
if outcome == "YES":
    outcome_type = "FULL_WIN" if entry < 0.5 else "LOSS"
elif outcome == "NO":
    outcome_type = "FULL_WIN" if entry > 0.5 else "LOSS"
else:
    outcome_type = "PARTIAL_WIN" if profitable else "LOSS"

# Calculate returns
if entry and entry > 0:
    exit_return_pct = round((exit_price - entry) / entry * 100, 2)
else:
    exit_return_pct = 0.0

# Peak: if it was a win, peak = exit. If loss, peak = entry (we never
# This is conservative but honest - real peaks need price history.
peak_price = exit_price if outcome_type in ("FULL_WIN", "PARTIAL_WIN")
peak_return_pct = round((peak_price - entry) / entry * 100, 2) if ent

await conn.execute("""
    UPDATE alerts
    SET outcome_type    = $1,
        entry_price     = $2,
        exit_price      = $3,
        exit_return_pct = $4,
        peak_price      = $5,
        peak_return_pct = $6,
        exit_reason     = 'RESOLVED',
        is_backfilled   = TRUE
    WHERE id = $7
""", outcome_type, entry, exit_price,
    exit_return_pct, peak_price, peak_return_pct,
    row["id"])
updated += 1

```

```

except Exception as e:
    log.warning("Backfill error on alert id=%s: %s", row["id"], e)

log.info("Backfill complete: %d alerts updated", updated)

async def init_db(conn):
    await conn.execute("""
        CREATE TABLE IF NOT EXISTS alerts (
            id SERIAL PRIMARY KEY,
            market_id TEXT NOT NULL,
            question TEXT NOT NULL,
            yes_price FLOAT NOT NULL,
            score INTEGER NOT NULL,
            reason TEXT NOT NULL,
            volume FLOAT NOT NULL,
            alerted_at TIMESTAMP NOT NULL,
            outcome TEXT,
            profitable BOOLEAN,
            user_rating TEXT,
            fear_greed_score INTEGER,
            fear_greed_regime TEXT,
            market_age_hours FLOAT,
            score_components TEXT,
            confidence_tier TEXT,
            category TEXT,
            -- Enhanced outcome tracking
            outcome_type TEXT,
            entry_price FLOAT,
            peak_price FLOAT,
            exit_price FLOAT,
            peak_return_pct FLOAT,
            exit_return_pct FLOAT,
            exit_reason TEXT,
            signals_fired TEXT,
            -- Rich data collection for backtest & CLOB strategy development
            days_to_resolution FLOAT,
            price_7d_ago FLOAT,
            price_3d_ago FLOAT,
            price_1d_ago FLOAT,
            score_breakdown TEXT,
            bid_price FLOAT,
            ask_price FLOAT,
            alerts_in_last_24h INTEGER,
            active_open_positions_count INTEGER,
            peak_reached_at TIMESTAMP,
            first_profitable_at TIMESTAMP,
            last_profitable_at TIMESTAMP
    """)

```

```

    )
    """)

# Trade position tracking - monitors open alerts for take-profit / stop-loss
await conn.execute("""
    CREATE TABLE IF NOT EXISTS trade_positions (
        id SERIAL PRIMARY KEY,
        alert_id INTEGER NOT NULL REFERENCES alerts(id),
        market_id TEXT NOT NULL,
        entry_price FLOAT NOT NULL,
        peak_price FLOAT NOT NULL,
        current_price FLOAT NOT NULL,
        opened_at TIMESTAMP NOT NULL,
        closed_at TIMESTAMP,
        exit_reason TEXT,
        exit_price FLOAT,
        return_pct FLOAT,
        outcome_type TEXT,
        is_open BOOLEAN DEFAULT TRUE
    )
    """)

# Paper trading simulation - resets daily, $200 starting bankroll
await conn.execute("""
    CREATE TABLE IF NOT EXISTS sim_trades (
        id SERIAL PRIMARY KEY,
        sim_date TEXT NOT NULL,
        alert_id INTEGER REFERENCES alerts(id),
        market_id TEXT NOT NULL,
        question TEXT NOT NULL,
        category TEXT,
        confidence_tier TEXT,
        entry_price FLOAT NOT NULL,
        stake FLOAT NOT NULL,
        exit_price FLOAT,
        return_pct FLOAT,
        pnl FLOAT,
        outcome_type TEXT,
        exit_reason TEXT,
        opened_at TIMESTAMP NOT NULL,
        closed_at TIMESTAMP,
        is_open BOOLEAN DEFAULT TRUE
    )
    """)

await conn.execute("""
    CREATE TABLE IF NOT EXISTS sim_daily_log (
        id SERIAL PRIMARY KEY,

```

```

        sim_date TEXT NOT NULL UNIQUE,
        starting_bankroll FLOAT NOT NULL DEFAULT 200.0,
        ending_bankroll FLOAT,
        total_staked FLOAT DEFAULT 0,
        total_pnl FLOAT DEFAULT 0,
        trades_placed INTEGER DEFAULT 0,
        trades_won INTEGER DEFAULT 0,
        trades_lost INTEGER DEFAULT 0,
        busted BOOLEAN DEFAULT FALSE,
        busted_at TIMESTAMP,
        created_at TIMESTAMP NOT NULL
    )
    """
)

# Dense price tracking for 48h after each alert – reveals optimal exit window
await conn.execute("""
    CREATE TABLE IF NOT EXISTS alert_price_snapshots (
        id SERIAL PRIMARY KEY,
        alert_id INTEGER REFERENCES alerts(id),
        market_id TEXT NOT NULL,
        yes_price FLOAT NOT NULL,
        bid_price FLOAT,
        ask_price FLOAT,
        recorded_at TIMESTAMP NOT NULL,
        minutes_since_alert INTEGER
    )
    """
)

await conn.execute("""
    CREATE TABLE IF NOT EXISTS opportunities_log (
        id SERIAL PRIMARY KEY,
        market_id TEXT NOT NULL,
        question TEXT NOT NULL,
        yes_price FLOAT NOT NULL,
        score INTEGER NOT NULL,
        reason TEXT NOT NULL,
        volume FLOAT NOT NULL,
        logged_at TIMESTAMP NOT NULL,
        outcome TEXT,
        profitable BOOLEAN,
        fear_greed_score INTEGER,
        fear_greed_regime TEXT,
        market_age_hours FLOAT,
        category TEXT
    )
    """
)

await conn.execute("""

```

```

CREATE TABLE IF NOT EXISTS price_history (
    id SERIAL PRIMARY KEY,
    market_id TEXT NOT NULL,
    yes_price FLOAT NOT NULL,
    recorded_at TIMESTAMP NOT NULL
)
)
)
await conn.execute("""
    CREATE TABLE IF NOT EXISTS sentiment_history (
        id SERIAL PRIMARY KEY,
        score INTEGER NOT NULL,
        classification TEXT NOT NULL,
        regime TEXT NOT NULL,
        trend TEXT,
        recorded_at TIMESTAMP NOT NULL
    )
)
)
await conn.execute("""
    CREATE TABLE IF NOT EXISTS risk_log (
        id SERIAL PRIMARY KEY,
        date TEXT NOT NULL,
        daily_loss FLOAT DEFAULT 0,
        trades_today INTEGER DEFAULT 0,
        win_streak INTEGER DEFAULT 0,
        loss_streak INTEGER DEFAULT 0,
        trade_size_multiplier FLOAT DEFAULT 1.0
    )
)
)
await conn.execute("""
    CREATE TABLE IF NOT EXISTS event_calendar (
        id SERIAL PRIMARY KEY,
        event_name TEXT NOT NULL,
        event_date TIMESTAMP NOT NULL,
        category TEXT NOT NULL,
        relevance_keywords TEXT NOT NULL,
        created_at TIMESTAMP NOT NULL
    )
)
)
)

# Safe column additions for existing tables
for col, typedef in [
    ("user_rating", "TEXT"),
    ("fear_greed_score", "INTEGER"),
    ("fear_greed_regime", "TEXT"),
    ("market_age_hours", "FLOAT"),
    ("score_components", "TEXT"),
    ("confidence_tier", "TEXT"),

```

```

        ("category", "TEXT"),
# Enhanced outcome tracking columns
        ("outcome_type", "TEXT"),
        ("entry_price", "FLOAT"),
        ("peak_price", "FLOAT"),
        ("exit_price", "FLOAT"),
        ("peak_return_pct", "FLOAT"),
        ("exit_return_pct", "FLOAT"),
        ("exit_reason", "TEXT"),
        ("signals_fired", "TEXT"),
        ("is_backfilled", "BOOLEAN"),
        ("days_to_resolution", "FLOAT"),
        ("price_7d_ago", "FLOAT"),
        ("price_3d_ago", "FLOAT"),
        ("price_1d_ago", "FLOAT"),
        ("score_breakdown", "TEXT"),
        ("bid_price", "FLOAT"),
        ("ask_price", "FLOAT"),
        ("alerts_in_last_24h", "INTEGER"),
        ("active_open_positions_count", "INTEGER"),
        ("peak_reached_at", "TIMESTAMP"),
        ("first_profitable_at", "TIMESTAMP"),
        ("last_profitable_at", "TIMESTAMP"),
]:
    await conn.execute(f"ALTER TABLE alerts ADD COLUMN IF NOT EXISTS {col} {t

for col, typedef in [("category", "TEXT")]:
    await conn.execute(f"ALTER TABLE opportunities_log ADD COLUMN IF NOT EXIS

await conn.execute("ALTER TABLE sentiment_history ADD COLUMN IF NOT EXISTS tr

log.info("Database tables ready")

# Backfill any resolved trades missing outcome_type / return data
await backfill_outcome_types(conn)

async def get_risk_state(conn):
    today = now().strftime("%Y-%m-%d")
    row = await conn.fetchrow("SELECT * FROM risk_log WHERE date = $1", today)
    if not row:
        # Carry over streaks from yesterday
        yesterday = await conn.fetchrow("""
            SELECT win_streak, loss_streak, trade_size_multiplier
            FROM risk_log ORDER BY date DESC LIMIT 1
        """)
        win_streak = yesterday["win_streak"] if yesterday else 0
        loss_streak = yesterday["loss_streak"] if yesterday else 0

```

```

        multiplier = yesterday["trade_size_multiplier"] if yesterday else 1.0
        await conn.execute("""
            INSERT INTO risk_log (date, daily_loss, trades_today, win_streak, los
            VALUES ($1, 0, 0, $2, $3, $4)
            """, today, win_streak, loss_streak, multiplier)
        row = await conn.fetchrow("SELECT * FROM risk_log WHERE date = $1", today)
        return dict(row)

async def reset_loss_streak(conn):
    today = now().strftime("%Y-%m-%d")
    await conn.execute("""
        UPDATE risk_log SET loss_streak = 0, win_streak = 0, trade_size_multiplie
        WHERE date = $1
        """, today)
    log.info("Loss streak reset to 0")

async def update_risk_state(conn, profitable):
    today = now().strftime("%Y-%m-%d")
    state = await get_risk_state(conn)
    if profitable:
        new_win = state["win_streak"] + 1
        new_loss = 0
        multiplier = 1.25 if new_win >= 3 else 1.0
    else:
        new_win = 0
        new_loss = state["loss_streak"] + 1
        multiplier = 0.75 if new_loss >= 3 else 1.0
    await conn.execute("""
        UPDATE risk_log SET win_streak=$1, loss_streak=$2, trade_size_multiplier=
        WHERE date=$4
        """, new_win, new_loss, multiplier, today)

def get_dynamic_limits(state):
    multiplier = state.get("trade_size_multiplier", 1.0)
    return {
        "max_trade_size": round(5 * multiplier, 2),
        "max_daily_loss": 20,
        "max_open_positions": 10
    }

async def log_alert(conn, opportunity, fear_greed=None, market_age=None):
    fg_score = fear_greed.get("score") if fear_greed else None
    fg_regime = fear_greed.get("regime") if fear_greed else None
    entry_price = opportunity["yes_price"]
    signals_fired = opportunity.get("signals_fired", "")

    # Count context metrics

```



```

alerts_24h = await conn.fetchval("""
    SELECT COUNT(*) FROM alerts
    WHERE alerted_at > NOW() - INTERVAL '24 hours'
    """)
open_positions = await conn.fetchval("""
    SELECT COUNT(*) FROM trade_positions WHERE is_open = TRUE
    """)

import json as _json
score_breakdown_str = None
if opportunity.get("score_breakdown"):
    try:
        score_breakdown_str = _json.dumps(opportunity["score_breakdown"])
    except Exception:
        pass

alert_id = await conn.fetchval("""
    INSERT INTO alerts (market_id, question, yes_price, score, reason, volume
                        fear_greed_score, fear_greed_regime, market_age_hours,
                        score_components, confidence_tier, category,
                        entry_price, peak_price, signals_fired,
                        days_to_resolution, price_7d_ago, price_3d_ago, price_
                        score_breakdown, bid_price, ask_price,
                        alerts_in_last_24h, active_open_positions_count)
    VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15,$16,
            $17,$18,$19,$20,$21,$22,$23,$24,$25)
    RETURNING id
    """,
    opportunity["id"], opportunity["question"], opportunity["yes_price"],
    opportunity["score"], opportunity["reason"], opportunity["volume"], now()
    fg_score, fg_regime, market_age,
    opportunity["reason"], opportunity.get("confidence_tier", "Medium"),
    opportunity.get("category", "General"),
    entry_price, entry_price,
    signals_fired,
    opportunity.get("days_to_resolution"),
    opportunity.get("price_7d_ago"),
    opportunity.get("price_3d_ago"),
    opportunity.get("price_1d_ago"),
    score_breakdown_str,
    opportunity.get("bid_price"),
    opportunity.get("ask_price"),
    alerts_24h,
    open_positions
    )
# Open a trade position for this alert so we can monitor it
await conn.execute("""

```

```

        INSERT INTO trade_positions (alert_id, market_id, entry_price, peak_price
                                    current_price, opened_at)
        VALUES ($1, $2, $3, $4, $5, $6)
        """ , alert_id, opportunity["id"], entry_price, entry_price, entry_price, now(
count = await conn.fetchval("SELECT COUNT(*) FROM alerts")
log.info("Alert logged (total: %d)", count)
return alert_id

async def log_opportunity(conn, opportunity, fear_greed=None, market_age=None):
    existing = await conn.fetchrow(
        "SELECT id FROM opportunities_log WHERE market_id=$1", opportunity["id"]
    )
    if existing:
        return
    fg_score = fear_greed.get("score") if fear_greed else None
    fg_regime = fear_greed.get("regime") if fear_greed else None
    await conn.execute("""
        INSERT INTO opportunities_log (market_id, question, yes_price, score, rea
                                    volume, logged_at, fear_greed_score, fear_g
                                    market_age_hours, category)
        VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11)
        """ ,
        opportunity["id"], opportunity["question"], opportunity["yes_price"],
        opportunity["score"], opportunity["reason"], opportunity["volume"], now()
        fg_score, fg_regime, market_age, opportunity.get("category", "General")
    )

async def update_price_history(conn, market_id, yes_price):
    last = await conn.fetchrow("""
        SELECT recorded_at FROM price_history
        WHERE market_id=$1 ORDER BY recorded_at DESC LIMIT 1
        """, market_id)
    if last:
        hours_since = (now() - last["recorded_at"]).total_seconds() / 3600
        if hours_since < 2:
            return
    await conn.execute("""
        INSERT INTO price_history (market_id, yes_price, recorded_at) VALUES ($1,
        """, market_id, yes_price, now())

async def get_price_history(conn, market_id, limit=20):
    rows = await conn.fetch("""
        SELECT yes_price, recorded_at FROM price_history
        WHERE market_id=$1 ORDER BY recorded_at DESC LIMIT $2
        """, market_id, limit)
    return rows

```

```

async def log_sentiment(conn, fear_greed):
    await conn.execute("""
        INSERT INTO sentiment_history (score, classification, regime, trend, reco
        VALUES ($1,$2,$3,$4,$5)
        """, fear_greed["score"], fear_greed["classification"],
        fear_greed["regime"], fear_greed.get("trend", "UNKNOWN"), now())

async def get_daily_stats(conn):
    return await conn.fetch("""
        SELECT * FROM alerts WHERE alerted_at > NOW() - INTERVAL '24 hours'
        """)

async def get_alerted_markets(conn):
    rows = await conn.fetch("SELECT DISTINCT market_id FROM alerts")
    return set(r["market_id"] for r in rows)

async def get_logged_opportunities(conn):
    rows = await conn.fetch("SELECT DISTINCT market_id FROM opportunities_log")
    return set(r["market_id"] for r in rows)

async def update_open_positions(conn):
    """
    Called every scan. For each open trade position:
    - Fetches current market price
    - Updates peak_price if price has risen
    - Closes position with TAKE_PROFIT if return >= +40%
    - Closes position with STOP_LOSS if return <= -25%
    Returns list of closed positions for Telegram notification.
    """
    import aiohttp, json as _json

    open_positions = await conn.fetch("""
        SELECT tp.*, a.question, a.entry_price as alert_entry
        FROM trade_positions tp
        JOIN alerts a ON tp.alert_id = a.id
        WHERE tp.is_open = TRUE
        AND tp.opened_at < NOW() - INTERVAL '10 minutes'
        """)

    if not open_positions:
        return []

    TAKE_PROFIT_PCT = 40.0    # close at +40% gain
    STOP_LOSS_PCT    = -25.0  # close at -25% loss

    closed = []
    headers = {

```

```

    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
    "Accept": "application/json"
}

async with aiohttp.ClientSession() as session:
    for pos in open_positions:
        try:
            url = "https://gamma-api.polymarket.com/markets?id=" + str(pos["m
            async with session.get(url, headers=headers,
                                   timeout=aiohttp.ClientTimeout(total=10)) a
            if resp.status != 200:
                continue
            markets = await resp.json()
            if not markets:
                continue
            market = markets[0]

            # Pull current price
            outcomes = market.get("outcomePrices", [])
            if isinstance(outcomes, str):
                outcomes = _json.loads(outcomes)
            if not outcomes:
                continue
            current_price = float(outcomes[0])

            entry = pos["entry_price"]
            if entry == 0:
                continue

            return_pct = round((current_price - entry) / entry * 100, 2)
            new_peak = max(pos["peak_price"], current_price)

            # Update peak and current price regardless
            await conn.execute("""
                UPDATE trade_positions
                SET current_price=$1, peak_price=$2
                WHERE id=$3
            """, current_price, new_peak, pos["id"])

            # Also keep alerts.peak_price fresh
            peak_return = round((new_peak - entry) / entry * 100, 2)
            await conn.execute("""
                UPDATE alerts SET peak_price=$1, peak_return_pct=$2
                WHERE id=$3
            """, new_peak, peak_return, pos["alert_id"])

            # Track timing - when peak hit, when first/last profitable

```

```

await update_position_timing(conn, pos["alert_id"], current_p

# Check market resolved
market_closed = market.get("closed", False)
exit_reason = None
outcome_type = None

if market_closed:
    exit_reason = "RESOLVED"
    if current_price >= 0.99:
        outcome_type = "FULL_WIN" if entry < 0.5 else "LOSS"
    elif current_price <= 0.01:
        outcome_type = "LOSS" if entry < 0.5 else "FULL_WIN"
    else:
        outcome_type = "PARTIAL_WIN" if return_pct > 5 else (
            "BREAKEVEN" if return_pct > -5 else "LOSS"
        )
elif return_pct >= TAKE_PROFIT_PCT:
    exit_reason = "TAKE_PROFIT"
    outcome_type = "PARTIAL_WIN"
elif return_pct <= STOP_LOSS_PCT:
    exit_reason = "STOP_LOSS"
    outcome_type = "LOSS"

if exit_reason:
    profitable = outcome_type in ("FULL_WIN", "PARTIAL_WIN")
    await conn.execute("""
        UPDATE trade_positions
        SET is_open=FALSE, closed_at=$1, exit_reason=$2,
            exit_price=$3, return_pct=$4, outcome_type=$5
        WHERE id=$6
    """, now(), exit_reason, current_price, return_pct, outco

    await conn.execute("""
        UPDATE alerts
        SET outcome=$1, profitable=$2, outcome_type=$3,
            exit_price=$4, exit_return_pct=$5, exit_reason=$6
        WHERE id=$7
    """,
        exit_reason, profitable, outcome_type,
        current_price, return_pct, exit_reason,
        pos["alert_id"]
    )

    await update_risk_state(conn, profitable)
    closed.append({
        "question": pos["question"],

```

```

        "entry_price": entry,
        "exit_price": current_price,
        "return_pct": return_pct,
        "peak_return_pct": peak_return,
        "exit_reason": exit_reason,
        "outcome_type": outcome_type,
        "profitable": profitable,
    })

    # Mirror close in simulation
    sim_result = await close_sim_trade(
        conn, pos["market_id"], current_price, exit_reason, o
    )
    if sim_result:
        closed[-1]["sim_result"] = sim_result
    log.info(
        "Position closed [%s] %s | entry=%.2f exit=%.2f retur
        outcome_type, pos["question"][:40],
        entry, current_price, return_pct
    )

    await asyncio.sleep(0.3)
except Exception as e:
    log.error("update_open_positions error: %s", e)

return closed

async def check_resolutions(conn):
    """
    Legacy full-resolution check (runs on schedule).
    Catches any markets that closed without being caught by update_open_positions
    """
    import aiohttp, json as _json

    unresolved_alerts = await conn.fetch("""
        SELECT * FROM alerts WHERE outcome IS NULL
        AND alerted_at < NOW() - INTERVAL '1 hour'
    """)
    unresolved_opps = await conn.fetch("""
        SELECT * FROM opportunities_log WHERE outcome IS NULL
        AND logged_at < NOW() - INTERVAL '1 hour'
    """)
    all_unresolved = list(unresolved_alerts) + list(unresolved_opps)
    if not all_unresolved:
        log.info("No unresolved items to check")
        return 0

```

```

log.info("Checking resolution for %d items", len(all_unresolved))
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.
    "Accept": "application/json"
}
resolved_count = 0
async with aiohttp.ClientSession() as session:
    for alert in all_unresolved:
        try:
            url = "https://gamma-api.polymarket.com/markets?id=" + str(alert[
            async with session.get(url, headers=headers,
                                timeout=aiohttp.ClientTimeout(total=10)) a
            if resp.status == 200:
                markets = await resp.json()
                if markets:
                    market = markets[0]
                    if market.get("closed", False):
                        outcomes = market.get("outcomePrices", "[ ]")
                        if isinstance(outcomes, str):
                            outcomes = _json.loads(outcomes)
                        if outcomes:
                            final_yes = float(outcomes[0])
                            entry = float(alert.get("entry_price") or ale
                            return_pct = round((final_yes - entry) / entr

                    if final_yes >= 0.99:
                        outcome = "YES"
                        outcome_type = "FULL_WIN" if entry < 0.5
                    elif final_yes <= 0.01:
                        outcome = "NO"
                        outcome_type = "LOSS" if entry < 0.5 else
                    else:
                        outcome = "PARTIAL"
                        outcome_type = (
                            "PARTIAL_WIN" if return_pct > 5
                            else "BREAKEVEN" if return_pct > -5
                            else "LOSS"
                        )

                profitable = outcome_type in ("FULL_WIN", "PA
                table = "alerts" if "alerted_at" in alert.key

                if table == "alerts":
                    await conn.execute("""
                        UPDATE alerts
                        SET outcome=$1, profitable=$2, outcom

```

```

        exit_price=$4, exit_return_pct=$5
        WHERE id=$6
        """, outcome, profitable, outcome_type,
        final_yes, return_pct, alert["id"])
        await update_risk_state(conn, profitable)
    else:
        await conn.execute(
            "UPDATE opportunities_log SET outcome
            outcome, profitable, alert['id']
            )

        resolved_count += 1
        log.info("Resolved: %s -> %s [%s] return=%.1f
            alert["question"][:40], outcome, out

        await asyncio.sleep(0.3)
    except Exception as e:
        log.error("Resolution error: %s", e)
    return resolved_count

async def run_weekly_backtest(conn):
    """
    Runs every Sunday. Analyses closed positions to find which signals,
    categories, confidence tiers, and fear/greed regimes actually win.
    Returns a formatted summary string for Telegram.
    """
    rows = await conn.fetch("""
        SELECT category, confidence_tier, fear_greed_regime, signals_fired,
            outcome_type, exit_return_pct, peak_return_pct, profitable
        FROM alerts
        WHERE outcome IS NOT NULL
        AND alerted_at > NOW() - INTERVAL '90 days'
        """)

    if not rows:
        return "Weekly Backtest: Not enough data yet. Keep building history!"

    total = len(rows)
    wins = sum(1 for r in rows if r["profitable"])
    win_rate = round(wins / total * 100, 1) if total else 0

    live_rows = [r for r in rows if not r.get("is_backfilled")]
    returns = [r["exit_return_pct"] for r in live_rows if r["exit_return_pct"] is
    avg_return = round(sum(returns) / len(returns), 1) if returns else None

    peak_returns = [r["peak_return_pct"] for r in live_rows if r["peak_return_pct"]
    avg_peak = round(sum(peak_returns) / len(peak_returns), 1) if peak_returns el

```



```

avg_return_str = ("+" if avg_return >= 0 else "") + str(avg_return) + "%"
avg_peak_str = ("+" + str(avg_peak) + "%") if avg_peak is not None else "trac

# Win rate by category
by_category = {}
for r in rows:
    cat = r["category"] or "Unknown"
    by_category.setdefault(cat, {"wins": 0, "total": 0})
    by_category[cat]["total"] += 1
    if r["profitable"]:
        by_category[cat]["wins"] += 1

cat_lines = []
for cat, d in sorted(by_category.items(), key=lambda x: -x[1]["total"]):
    wr = round(d["wins"] / d["total"] * 100, 1) if d["total"] else 0
    cat_lines.append(f" {cat}: {wr}% ({d['total']} trades)")

# Win rate by confidence tier
by_tier = {}
for r in rows:
    tier = r["confidence_tier"] or "Unknown"
    by_tier.setdefault(tier, {"wins": 0, "total": 0})
    by_tier[tier]["total"] += 1
    if r["profitable"]:
        by_tier[tier]["wins"] += 1

tier_lines = []
for tier in ["HIGH", "MEDIUM", "LOW"]:
    if tier in by_tier:
        d = by_tier[tier]
        wr = round(d["wins"] / d["total"] * 100, 1)
        tier_lines.append(f" {tier}: {wr}% ({d['total']} trades)")

# Top signals analysis
signal_stats = {}
for r in rows:
    sigs = r["signals_fired"] or ""
    for sig in sigs.split(","):
        sig = sig.strip()
        if not sig:
            continue
        signal_stats.setdefault(sig, {"wins": 0, "total": 0})
        signal_stats[sig]["total"] += 1
        if r["profitable"]:
            signal_stats[sig]["wins"] += 1

```

```

sig_lines = []
for sig, d in sorted(signal_stats.items(), key=lambda x: -x[1]["total"])[0:6]:
    wr = round(d["wins"] / d["total"] * 100, 1) if d["total"] else 0
    sig_lines.append(f"  {sig}: {wr}% ({d['total']} trades)")

# Outcome type breakdown
outcome_counts = {}
for r in rows:
    ot = r["outcome_type"] or "UNKNOWN"
    outcome_counts[ot] = outcome_counts.get(ot, 0) + 1

outcome_lines = [f"  {k}: {v}" for k, v in sorted(outcome_counts.items(), key

sig_section = sig_lines if sig_lines else ["  No signal data yet"]

lines = [
    " <b>Weekly Backtest Report</b>",
    "",
    f"<b>Overall ({total} closed trades)</b>",
    f"Win Rate: {win_rate}%",
    f"Avg Exit Return: {avg_return_str}",
    f"Avg Peak Return: {avg_peak_str}",
    "",
    "<b>By Category:</b>",
] + cat_lines + [
    "",
    "<b>By Confidence Tier:</b>",
] + tier_lines + [
    "",
    "<b>By Signal:</b>",
] + sig_section + [
    "",
    "<b>Outcome Breakdown:</b>",
] + outcome_lines

return "\n".join(lines)

# — SIMULATION ENGINE —————

SIM_DAILY_BUDGET      = 200.0
SIM_STAKE_HIGH        = 40.0   # HIGH confidence alerts
SIM_STAKE_MEDIUM      = 25.0   # MEDIUM confidence alerts
SIM_STAKE_LOW         = 15.0   # LOW confidence alerts
SIM_TAKE_PROFIT_PCT   = 40.0   # mirror real position management
SIM_STOP_LOSS_PCT     = -25.0

```

```

async def get_sim_state(conn):
    """
    Returns today's simulation row, creating it if this is the first
    alert of the day. Bankroll always starts at $200.
    """
    today = now().strftime("%Y-%m-%d")
    row = await conn.fetchrow("SELECT * FROM sim_daily_log WHERE sim_date = $1",
    if not row:
        await conn.execute("""
            INSERT INTO sim_daily_log
                (sim_date, starting_bankroll, ending_bankroll, total_staked,
                 total_pnl, trades_placed, trades_won, trades_lost,
                 busted, created_at)
            VALUES ($1, $2, $2, 0, 0, 0, 0, 0, FALSE, $3)
        """, today, SIM_DAILY_BUDGET, now())
        row = await conn.fetchrow("SELECT * FROM sim_daily_log WHERE sim_date = $
    return dict(row)

async def sim_is_active(conn):
    """Returns False if today's sim has busted (bankroll at $0)."""
    state = await get_sim_state(conn)
    return not state["busted"] and (state["ending_bankroll"] or 0) > 0

async def place_sim_trade(conn, alert_id, opp):
    """
    Called when a real alert fires. Places a hypothetical stake if the
    sim is still active today. Stake size is determined by confidence tier.
    Returns the sim trade row id, or None if sim is busted/skipped.
    """
    if not await sim_is_active(conn):
        return None

    today = now().strftime("%Y-%m-%d")
    state = await get_sim_state(conn)
    bankroll = state["ending_bankroll"] or 0.0

    tier = opp.get("confidence_tier", "MEDIUM").upper()
    raw_stake = {
        "HIGH": SIM_STAKE_HIGH,
        "MEDIUM": SIM_STAKE_MEDIUM,
        "LOW": SIM_STAKE_LOW,
    }.get(tier, SIM_STAKE_MEDIUM)

    stake = min(raw_stake, bankroll) # never bet more than we have
    if stake <= 0:

```

```

        return None

    new_bankroll = round(bankroll - stake, 2)

    sim_id = await conn.fetchval("""
        INSERT INTO sim_trades
            (sim_date, alert_id, market_id, question, category,
             confidence_tier, entry_price, stake, opened_at)
        VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9)
        RETURNING id
    """, today, alert_id, opp["id"], opp["question"][:200],
        opp.get("category", "General"), tier,
        opp["yes_price"], stake, now())

    await conn.execute("""
        UPDATE sim_daily_log
        SET ending_bankroll = $1,
            total_staked     = total_staked + $2,
            trades_placed    = trades_placed + 1
        WHERE sim_date = $3
    """, new_bankroll, stake, today)

    log.info("SIM trade placed | %s | stake=$%.2f | bankroll=$%.2f",
            tier, stake, new_bankroll)
    return sim_id

async def close_sim_trade(conn, market_id, current_price, exit_reason, outcome_ty
    """
    Called whenever a real trade_position closes (take-profit, stop-loss,
    or resolution). Finds the matching open sim trade, calculates P&L,
    updates bankroll, and checks for bust.
    Returns dict with result info, or None if no open sim trade found.
    """
    today = now().strftime("%Y-%m-%d")

    sim_trade = await conn.fetchrow("""
        SELECT * FROM sim_trades
        WHERE market_id = $1 AND is_open = TRUE AND sim_date = $2
    """, market_id, today)

    if not sim_trade:
        return None

    entry  = sim_trade["entry_price"]
    stake  = sim_trade["stake"]
    return_pct = round((current_price - entry) / entry * 100, 2) if entry else 0

```

```

pnl      = round(stake * return_pct / 100, 2)
profitable = outcome_type in ("FULL_WIN", "PARTIAL_WIN")

await conn.execute("""
    UPDATE sim_trades
    SET exit_price  = $1,
        return_pct  = $2,
        pnl         = $3,
        outcome_type = $4,
        exit_reason  = $5,
        closed_at    = $6,
        is_open      = FALSE
    WHERE id = $7
    """, current_price, return_pct, pnl, outcome_type, exit_reason, now(), sim_tr

# Update daily bankroll
state = await get_sim_state(conn)
new_bankroll = round((state["ending_bankroll"] or 0) + stake + pnl, 2)
new_bankroll = max(0.0, new_bankroll)
busted = new_bankroll <= 0

win_delta  = 1 if profitable else 0
loss_delta = 0 if profitable else 1

await conn.execute("""
    UPDATE sim_daily_log
    SET ending_bankroll = $1,
        total_pnl       = total_pnl + $2,
        trades_won      = trades_won + $3,
        trades_lost     = trades_lost + $4,
        busted          = $5,
        busted_at       = CASE WHEN $5 THEN $6 ELSE busted_at END
    WHERE sim_date = $7
    """, new_bankroll, pnl, win_delta, loss_delta, busted, now(), today)

if busted:
    log.warning("SIM BUSTED today - bankroll hit $0. Monitoring continues.")

log.info("SIM closed [%s] return=%.1f%% pnl=$%.2f bankroll=$%.2f%s",
        outcome_type, return_pct, pnl, new_bankroll, " BUSTED" if busted else

return {
    "sim_trade_id": sim_trade["id"],
    "stake": stake,
    "return_pct": return_pct,
    "pnl": pnl,
    "new_bankroll": new_bankroll,

```

```

        "outcome_type": outcome_type,
        "exit_reason": exit_reason,
        "busted": busted,
        "profitable": profitable,
        "question": sim_trade["question"],
    }

```

```

async def get_sim_summary(conn, sim_date=None):
    """Returns the sim_daily_log row for a given date (default: today)."""
    date_str = sim_date or now().strftime("%Y-%m-%d")
    row = await conn.fetchrow(
        "SELECT * FROM sim_daily_log WHERE sim_date = $1", date_str
    )
    return dict(row) if row else None

async def get_sim_trades_for_date(conn, sim_date=None):
    """Returns all sim trades for a given date."""
    date_str = sim_date or now().strftime("%Y-%m-%d")
    rows = await conn.fetch(
        "SELECT * FROM sim_trades WHERE sim_date = $1 ORDER BY opened_at", date_s
    )
    return [dict(r) for r in rows]

async def run_sim_weekly_report(conn):
    """
    Computes a 7-day simulation performance report.
    Called alongside the weekly backtest every Sunday.
    """
    rows = await conn.fetch("""
        SELECT * FROM sim_daily_log
        WHERE created_at > NOW() - INTERVAL '7 days'
        ORDER BY sim_date
    """)
    if not rows:
        return "📊 Sim Report: No data yet for the past 7 days."

    total_pnl = sum(r["total_pnl"] or 0 for r in rows)
    total_staked = sum(r["total_staked"] or 0 for r in rows)
    total_trades = sum(r["trades_placed"] or 0 for r in rows)
    total_wins = sum(r["trades_won"] or 0 for r in rows)
    bust_days = sum(1 for r in rows if r["busted"])
    win_rate = round(total_wins / total_trades * 100, 1) if total_trades else
    roi = round(total_pnl / total_staked * 100, 1) if total_staked else

```

```

day_lines = []
for r in rows:
    pnl      = r["total_pnl"] or 0
    br       = r["ending_bankroll"] or 0
    busted   = " 🤖 BUSTED" if r["busted"] else ""
    sign     = "+" if pnl >= 0 else ""
    day_lines.append(
        f" {r['sim_date']}: {sign}${round(pnl,2)} "
        f"({r['trades_won']}/{r['trades_placed']} wins) "
        f"→ ${round(br,2)}{busted}"
    )

sign = "+" if total_pnl >= 0 else ""
lines = [
    " 🎮 <b>Weekly Sim Report ($200/day)</b>",
    "",
    f"<b>7-Day Summary:</b>",
    f" Total P&L: {sign}${round(total_pnl, 2)}",
    f" Total Staked: ${round(total_staked, 2)}",
    f" ROI on staked: {sign}{roi}%",
    f" Win Rate: {win_rate}% ({total_wins}/{total_trades})",
    f" Bust Days: {bust_days}/7",
    "",
    "<b>Daily Breakdown:</b>",
] + day_lines

return "\n".join(lines)

```

— RICH DATA COLLECTION FUNCTIONS —————

```

async def record_alert_snapshot(conn, alert_id, market_id, yes_price,
                                alerted_at, bid_price=None, ask_price=None):
    """
    Records a price snapshot for an alerted market every 15 minutes
    for 48 hours after the alert fired. Builds the post-alert price
    curve needed to identify optimal exit windows.
    Only records if at least 14 minutes have passed since last snapshot.
    """
    last = await conn.fetchrow("""
        SELECT recorded_at FROM alert_price_snapshots
        WHERE alert_id = $1 ORDER BY recorded_at DESC LIMIT 1
        """, alert_id)

    if last:
        mins_since = (now() - last["recorded_at"]).total_seconds() / 60
        if mins_since < 14:
            return

```

```

# Stop snapshotting after 48 hours
hours_since_alert = (now() - alerted_at).total_seconds() / 3600
if hours_since_alert > 48:
    return

minutes_since = int((now() - alerted_at).total_seconds() / 60)

await conn.execute("""
    INSERT INTO alert_price_snapshots
        (alert_id, market_id, yes_price, bid_price, ask_price,
         recorded_at, minutes_since_alert)
    VALUES ($1,$2,$3,$4,$5,$6,$7)
""", alert_id, market_id, yes_price, bid_price, ask_price, now(), minutes_sin

async def update_position_timing(conn, alert_id, current_price, entry_price):
    """
    Called every scan for open positions. Tracks:
    - peak_reached_at: when the highest price was seen
    - first_profitable_at: first time price exceeded entry
    - last_profitable_at: most recent time price was above entry
    These three timestamps reveal the shape of the opportunity window.
    """
    alert = await conn.fetchrow("""
        SELECT peak_price, peak_reached_at, first_profitable_at, entry_price
        FROM alerts WHERE id = $1
    """, alert_id)

    if not alert:
        return

    updates = {}
    current_peak = alert["peak_price"] or entry_price

    # Update peak timing if new high
    if current_price > current_peak:
        updates["peak_reached_at"] = now()

    # Track profitable window
    if current_price > entry_price:
        if not alert["first_profitable_at"]:
            updates["first_profitable_at"] = now()
        updates["last_profitable_at"] = now()

    if updates:
        set_clauses = ", ".join(f"{k} = ${i+2}" for i, k in enumerate(updates))

```



```

values = list(updates.values())
await conn.execute(
    f"UPDATE alerts SET {set_clauses} WHERE id = $1",
    alert_id, *values
)

```

```

async def get_alert_price_curve(conn, alert_id):
    """Returns the full post-alert price curve for a given alert."""
    rows = await conn.fetch("""
        SELECT yes_price, bid_price, ask_price, recorded_at, minutes_since_alert
        FROM alert_price_snapshots
        WHERE alert_id = $1
        ORDER BY recorded_at ASC
    """, alert_id)
    return [dict(r) for r in rows]

```

```

async def cleanup_old_snapshots(conn):
    """
    Removes snapshots older than 30 days to keep the table lean.
    Called weekly.
    """
    deleted = await conn.fetchval("""
        DELETE FROM alert_price_snapshots
        WHERE recorded_at < NOW() - INTERVAL '30 days'
        RETURNING id
    """)
    if deleted:
        log.info("Cleaned up old price snapshots")

```